OS write buffering and page cache size

> Does OS buffers data before write those data to storage like ssd. If OS does do it. What's the size for buffering write data?

Yes, the OS buffers writes before flushing to storage. This is called the **page cache** (or buffer cache).

How it works

When you write data, it goes: **App → Page Cache (RAM) → Storage device**

The OS marks those pages as "dirty" and flushes them to disk asynchronously, so your app doesn't have to wait for slow storage I/O.

Buffer/Cache size

There's **no fixed size** — the page cache dynamically uses **all available free RAM**. The kernel grows and shrinks it on demand:

- Got 16 GB RAM and only 4 GB used by apps? The remaining ~12 GB can be page cache.
- When an app needs more RAM, the kernel evicts clean cached pages to make room.

Key Linux kernel tunables (on Linux)

Parameter	Default	Meaning
`vm.dirty_ratio`	20%	% of total RAM that can hold dirty (unwritten) data before writes **block**
`vm.dirty_background_ratio`	10%	% at which background flusher **starts** writing dirty pages
`vm.dirty_expire_centisecs`	3000 (30s)	How long a dirty page can sit before being flushed
`vm.dirty_writeback_centisecs`	500 (5s)	How often the flusher daemon wakes up

So on a **16 GB RAM** machine with defaults, up to **~3.2 GB** of dirty writes can accumulate before the OS starts forcing flushes.

Bypassing the buffer

If you need guaranteed durability (e.g., databases), you can bypass the cache:

- `fsync()` / `fdatasync()` — flush a specific file's dirty pages to disk
- `O_SYNC` / `O_DIRECT` flag — write directly to storage, bypassing the page cache entirely

This is why databases like PostgreSQL call `fsync()` after committing transactions — otherwise a power loss could lose buffered writes still in RAM.

I want use C language to write data into SSD. Can you teach me how to do it?

Here's how to write data to an SSD using C:

Basic File Write (Buffered — default)

c

```c
#include <stdio.h>
#include <stdlib.h>

int main() {
    FILE *fp = fopen("output.bin", "wb");
    if (!fp) {
        perror("fopen");
        return 1;
    }

    const char *data = "Hello, SSD!";
    size_t written = fwrite(data, 1, 11, fp);
    if (written != 11) {
        perror("fwrite");
    }

    fclose(fp);  // flushes stdio buffer + closes
    return 0;
}
```

This uses the **stdio library buffer** (typically 4–8 KB) on top of the OS page cache. Two layers of buffering.

Flushing to Guarantee Durability

c

```c
#include <stdio.h>
#include <unistd.h>

int main() {
    FILE *fp = fopen("output.bin", "wb");

    const char *data = "Critical data";
    fwrite(data, 1, 13, fp);

    fflush(fp);          // 1. Flush stdio buffer → OS page cache
    fsync(fileno(fp));   // 2. Flush OS page cache → SSD hardware

    fclose(fp);
    return 0;
}
```

- `fflush()` — pushes data from the C library buffer to the OS
- `fsync()` — forces the OS to write dirty pages all the way to the SSD

Direct I/O — Bypass OS Cache entirely (Linux)

```c
#define _GNU_SOURCE
#include <fcntl.h>
#include <unistd.h>
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

int main() {
    // O_DIRECT bypasses the OS page cache
    int fd = open("output.bin", O_WRONLY | O_CREAT | O_DIRECT, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // IMPORTANT: buffer must be aligned to block size (usually 512B or 4096B)
    size_t block_size = 4096;
    void *buf;
    posix_memalign(&buf, block_size, block_size);  // aligned allocation
    memset(buf, 0xAB, block_size);                 // fill with some data

    ssize_t written = write(fd, buf, block_size);
    if (written < 0) {
        perror("write");
```

```
    }

    free(buf);
    close(fd);
    return 0;
}
```

> **O_DIRECT rules:** buffer address, size, and file offset must all be aligned to the device's logical block size (usually **4096 bytes**). Misalignment causes `EINVAL`.

Comparison Summary

Method	Stdio Buffer	OS Page Cache	Durability	Speed
`fwrite` + `fclose`	✅	✅	❌ (power loss risk)	Fastest
`fwrite` + `fflush` + `fsync`	✅	✅ flushed	✅	Slower
`write` (syscall)	❌	✅	❌ (power loss risk)	Fast
`write` + `fsync`	❌	✅ flushed	✅	Slower
`O_DIRECT` + `write`	❌	❌ bypassed	✅	Varies*

*O_DIRECT can be faster for large sequential writes (databases), but slower for small random writes since you lose OS caching benefits.

Which should you use?

- **General use** → `fwrite` + `fclose` is fine
- **Database / critical data** → `write` + `fsync` after each transaction
- **High-performance storage engine** → `O_DIRECT` with careful alignment (like what PostgreSQL, MySQL, and RocksDB do)

By the way, I want to do flexible data placement to write data into SSD. Can you teach me how?

Great question! **Flexible Data Placement (FDP)** is a relatively new NVMe feature (introduced in NVMe 2.0 spec) that lets you hint to the SSD *where* to place data physically, reducing Write Amplification Factor (WAF).

What is FDP?

Normally the SSD's FTL (Flash Translation Layer) decides where to place data internally. With FDP you can group related data into **Reclaim Units** via **Placement Handles**, so the SSD can garbage collect them more efficiently.

```
Without FDP:  Hot + Cold data mixed → GC must move lots of live data → High WAF
With FDP:     Hot data → RG0,  Cold data → RG1 → GC only touches cold group → Low WAF
```

Prerequisites

- NVMe SSD that supports FDP (e.g., Samsung 990 Pro, some datacenter SSDs)
- Linux kernel **6.8+** (FDP support added)
- `nvme-cli` to inspect and configure FDP

Check if your SSD supports FDP

bash

```bash
nvme fdp status /dev/nvme0n1
nvme id-ns /dev/nvme0n1 --human-readable | grep -i fdp
```

Method 1: Using `io_uring` with FDP hints (Recommended)

The modern Linux way — uses `io_uring` with placement hints via `RWF_PLACEMENT`.

c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <liburing.h>      // apt install liburing-dev
#include <linux/io_uring.h>

#define BLOCK_SIZE  4096
#define QUEUE_DEPTH 4

int main() {
    struct io_uring ring;
    int ret;

    // 1. Init io_uring
```

```c
    ret = io_uring_queue_init(QUEUE_DEPTH, &ring, 0);
    if (ret < 0) {
        fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-ret));
        return 1;
    }

    // 2. Open file with O_DIRECT (required for FDP)
    int fd = open("/dev/nvme0n1", O_WRONLY | O_DIRECT, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // 3. Allocate aligned buffer
    void *buf;
    posix_memalign(&buf, BLOCK_SIZE, BLOCK_SIZE);
    memset(buf, 0xAB, BLOCK_SIZE);

    // 4. Get SQE and set FDP placement hint
    struct io_uring_sqe *sqe = io_uring_get_sqe(&ring);

    io_uring_prep_write(sqe, fd, buf, BLOCK_SIZE, 0);  // offset=0

    // Set placement handle (0 = first placement group, 1 = second, etc.)
    sqe->ioprio = IORING_RECVSEND_FIXED_BUF;
    sqe->file_index = 0;        // placement handle index
    sqe->addr3 = 1;             // Reclaim Group hint (RG ID)

    // 5. Submit
    io_uring_submit(&ring);

    // 6. Wait for completion
    struct io_uring_cqe *cqe;
    io_uring_wait_cqe(&ring, &cqe);
    if (cqe->res < 0) {
        fprintf(stderr, "write failed: %s\n", strerror(-cqe->res));
    } else {
        printf("Written %d bytes with FDP placement hint\n", cqe->res);
    }

    io_uring_cqe_seen(&ring, cqe);

    // Cleanup
    free(buf);
    close(fd);
    io_uring_queue_exit(&ring);
    return 0;
}
```

Method 2: Using `pwritev2` with `RWF_PLACEMENT` flag

A simpler syscall-based approach without io_uring.

c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/uio.h>
#include <linux/fs.h>

#ifndef RWF_PLACEMENT
#define RWF_PLACEMENT 0x00000200   // define if not in your headers yet
#endif

int main() {
    // Open the NVMe namespace directly with O_DIRECT
    int fd = open("/dev/nvme0n1", O_WRONLY | O_DIRECT, 0644);
    if (fd < 0) {
        perror("open");
        return 1;
    }

    // Aligned buffer
    void *buf;
    posix_memalign(&buf, 4096, 4096);
    memset(buf, 0xCD, 4096);

    struct iovec iov = {
        .iov_base = buf,
        .iov_len  = 4096,
    };

    // pwritev2 with placement hint
    // Last arg is flags; RWF_PLACEMENT tells kernel to use FDP placement
    ssize_t ret = pwritev2(fd, &iov, 1,
                           0,              // file offset
                           RWF_PLACEMENT); // FDP hint flag
    if (ret < 0) {
        perror("pwritev2");
    } else {
        printf("Written %zd bytes with FDP hint\n", ret);
    }
```

```
    free(buf);
    close(fd);
    return 0;
}
```

Method 3: Using `fio` to test FDP first (No coding needed)

Before writing code, validate your SSD supports FDP with `fio`:

bash

```
# Install fio with FDP support
apt install fio

# Test FDP write with placement hints
fio --name=fdp-test \
    --filename=/dev/nvme0n1 \
    --ioengine=io_uring \
    --rw=randwrite \
    --bs=4k \
    --fdp=1 \
    --fdp_pli=0,1,2 \       # placement handle indices to use
    --numjobs=1 \
    --iodepth=8 \
    --size=1g \
    --direct=1
```

FDP Concepts Summary

Term	Meaning
Placement Handle (PH)	A hint ID you attach to a write (0, 1, 2...)
Reclaim Unit (RU)	Physical erase unit on NAND flash
Reclaim Group (RG)	A group of RUs sharing the same placement
FTL	SSD firmware that maps logical → physical blocks

Typical Use Case — Separate Hot/Cold Data

c

```
#define PH_HOT  0   // frequently updated data → placement handle 0
#define PH_COLD 1   // rarely updated data     → placement handle 1
#define PH_META 2   // metadata                → placement handle 2
```

```
// Hot data write (logs, indexes)
write_with_placement(fd, hot_buf,  4096, offset_hot,  PH_HOT);

// Cold data write (archives, blobs)
write_with_placement(fd, cold_buf, 4096, offset_cold, PH_COLD);
```

By keeping hot and cold data in separate reclaim groups, the SSD's GC doesn't need to copy live cold data when erasing hot data blocks — dramatically reducing WAF.

Build & Run

bash

```
# Method 1 (io_uring)
gcc -o fdp_uring fdp_uring.c -luring

# Method 2 (pwritev2)
gcc -o fdp_pwrite fdp_pwrite.c

# Check kernel version (need 6.8+)
uname -r
```

Would you like me to show a more complete example with multiple placement handles writing hot/cold data in a loop?

Does OS page cache buffers FDP write data?

Good question! The answer is: **it depends on how you write.**

Short Answer

Write Method	Page Cache	FDP Hint Preserved?
`fwrite` / `write` (buffered)	✅ Yes	❌ Lost
`pwritev2` with `RWF_PLACEMENT`	✅ Yes	✅ Preserved in page cache
`O_DIRECT` + placement hint	❌ Bypassed	✅ Sent directly to SSD

Key Detail — `RWF_PLACEMENT` + Page Cache

When you use `pwritev2(..., RWF_PLACEMENT)` **without** `O_DIRECT`:

- Data **does** go into the page cache as dirty pages
- The kernel **tags those dirty pages** with the placement hint metadata
- When the page cache eventually flushes to the SSD, it **carries the FDP hint along** with the I/O request down to the NVMe driver

So the hint is **not lost** — it rides with the data through the flush path.

```
App
 |  pwritev2(..., RWF_PLACEMENT, PH=1)
 ▼
Page Cache  ← dirty page tagged with PH=1
 |  (flush via writeback)
 ▼
NVMe Driver ← FDP placement hint attached to I/O
 |
 ▼
SSD Physical NAND (Reclaim Group 1)
```

But There's a Catch

Since the page cache **batches and reorders** writes before flushing:

- **Timing is non-deterministic** — you don't control exactly when the hint reaches the SSD
- The kernel **may merge** adjacent page cache writes, potentially mixing placement hints
- For strict FDP control (databases, storage engines), `O_DIRECT` is strongly preferred so hints reach the SSD **immediately and intact**

Bottom Line

- `O_DIRECT` + **FDP** → Full control, hint goes straight to SSD, no buffering
- `RWF_PLACEMENT` **without** `O_DIRECT` → Hint is preserved through page cache flush, but you lose precise timing control
- **Regular buffered write** → Page cache has no concept of FDP, hint is gone entirely

This is exactly why production storage engines like **RocksDB** and **MySQL InnoDB** use `O_DIRECT` when implementing FDP support — they need deterministic placement, not best-effort.

Teach me how to sent data directly to SSD by FDP with page cache bypassed.

FDP Write with O_DIRECT (Page Cache Bypassed)

Here's a complete, well-structured example:

Step 1 — Check FDP Support & Get Placement Handles

bash

```bash
# Check FDP status
nvme fdp status /dev/nvme0n1

# List available placement handles
nvme fdp usage /dev/nvme0n1

# Check your kernel version (need 6.8+)
uname -r
```

Step 2 — Complete C Code

c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/uio.h>
#include <sys/ioctl.h>
#include <linux/nvme_ioctl.h>
#include <errno.h>

/* FDP placement handle IDs — map to your SSD's reclaim groups */
#define PH_HOT   0    /* hot data  : logs, indexes         */
#define PH_COLD  1    /* cold data : archives, blobs        */
#define PH_META  2    /* metadata  : superblocks, manifests */

#define BLOCK_SIZE  4096
#define ALIGN_SIZE  4096

/* RWF_PLACEMENT flag — define manually if your headers don't have it yet */
#ifndef RWF_PLACEMENT
#define RWF_PLACEMENT 0x00000200
#endif
```

```c
/* ---------------------------------------------------------------
 * encode_placement_hint
 * Encodes a placement handle into the flags field for pwritev2.
 * The upper 16 bits of rwf_flags carry the placement handle ID.
 * --------------------------------------------------------------- */
static inline int encode_placement_hint(int placement_handle)
{
    return RWF_PLACEMENT | (placement_handle << 16);
}

/* ---------------------------------------------------------------
 * open_nvme_direct
 * Opens an NVMe namespace with O_DIRECT to bypass page cache.
 * --------------------------------------------------------------- */
static int open_nvme_direct(const char *dev_path)
{
    int fd = open(dev_path, O_WRONLY | O_DIRECT);
    if (fd < 0) {
        perror("open");
        fprintf(stderr, "Failed to open device: %s\n", dev_path);
    }
    return fd;
}

/* ---------------------------------------------------------------
 * alloc_aligned_buf
 * Allocates a buffer aligned to ALIGN_SIZE.
 * O_DIRECT requires buffer address, size, and file offset
 * to all be aligned to the device logical block size (4096).
 * --------------------------------------------------------------- */
static void *alloc_aligned_buf(size_t size)
{
    void *buf = NULL;
    if (posix_memalign(&buf, ALIGN_SIZE, size) != 0) {
        perror("posix_memalign");
        return NULL;
    }
    return buf;
}

/* ---------------------------------------------------------------
 * fdp_write
 * Writes data directly to SSD at `offset` using FDP placement
 * handle `ph`. Page cache is bypassed via O_DIRECT.
 *
 * Args:
 *   fd      - file descriptor opened with O_DIRECT
 *   buf     - aligned buffer (must be ALIGN_SIZE aligned)
 *   size    - number of bytes to write (must be multiple of BLOCK_SIZE)
```

```c
 *   offset  - byte offset on device (must be multiple of BLOCK_SIZE)
 *   ph      - placement handle (PH_HOT, PH_COLD, PH_META, ...)
 *
 * Returns: bytes written on success, -1 on failure.
 * ------------------------------------------------------------------ */
static ssize_t fdp_write(int fd, void *buf, size_t size,
                         off_t offset, int ph)
{
    struct iovec iov = {
        .iov_base = buf,
        .iov_len  = size,
    };

    int flags = encode_placement_hint(ph);

    ssize_t ret = pwritev2(fd, &iov, 1, offset, flags);
    if (ret < 0) {
        fprintf(stderr, "fdp_write failed (ph=%d, offset=%ld): %s\n",
                ph, offset, strerror(errno));
    }
    return ret;
}

/* ------------------------------------------------------------------
 * demo: write hot, cold, and metadata blocks to separate
 *       reclaim groups on the SSD.
 * ------------------------------------------------------------------ */
int main(void)
{
    const char *dev = "/dev/nvme0n1";

    int fd = open_nvme_direct(dev);
    if (fd < 0) return 1;

    /* --- Allocate aligned buffers --- */
    void *hot_buf  = alloc_aligned_buf(BLOCK_SIZE);
    void *cold_buf = alloc_aligned_buf(BLOCK_SIZE);
    void *meta_buf = alloc_aligned_buf(BLOCK_SIZE);

    if (!hot_buf || !cold_buf || !meta_buf) {
        close(fd);
        return 1;
    }

    /* --- Fill buffers with example data --- */
    memset(hot_buf,  0xAA, BLOCK_SIZE);   /* hot  data pattern */
    memset(cold_buf, 0xBB, BLOCK_SIZE);   /* cold data pattern */
    memset(meta_buf, 0xCC, BLOCK_SIZE);   /* meta data pattern */
```

```
    /* --- Write each buffer to a different reclaim group --- */
    ssize_t ret;

    /* Hot data → Reclaim Group 0 (offset 0) */
    ret = fdp_write(fd, hot_buf, BLOCK_SIZE, 0 * BLOCK_SIZE, PH_HOT);
    if (ret > 0) printf("[HOT]  Written %zd bytes at offset 0 → PH=%d\n",
                        ret, PH_HOT);

    /* Cold data → Reclaim Group 1 (offset 4096) */
    ret = fdp_write(fd, cold_buf, BLOCK_SIZE, 1 * BLOCK_SIZE, PH_COLD);
    if (ret > 0) printf("[COLD] Written %zd bytes at offset 4096 → PH=%d\n",
                        ret, PH_COLD);

    /* Metadata → Reclaim Group 2 (offset 8192) */
    ret = fdp_write(fd, meta_buf, BLOCK_SIZE, 2 * BLOCK_SIZE, PH_META);
    if (ret > 0) printf("[META] Written %zd bytes at offset 8192 → PH=%d\n",
                        ret, PH_META);

    /* --- Cleanup --- */
    free(hot_buf);
    free(cold_buf);
    free(meta_buf);
    close(fd);

    return 0;
}
```

Step 3 — Build & Run

bash

```
# Compile
gcc -o fdp_direct fdp_direct.c

# Run (requires root — writing directly to NVMe device)
sudo ./fdp_direct
```

Step 4 — Verify FDP Usage

bash

```
# Check reclaim unit usage after writing
nvme fdp usage /dev/nvme0n1

# Monitor FDP statistics
nvme fdp stats /dev/nvme0n1
```

How the Data Flows

```
Your App
 |
 |  fdp_write(fd, buf, size, offset, PH_HOT)
 |  pwritev2(..., RWF_PLACEMENT | PH<<16)
 ▼
Kernel VFS
 |  O_DIRECT → page cache is BYPASSED
 |  placement hint encoded in I/O request
 ▼
NVMe Driver
 |  translates hint → NVMe Placement Handle field
 |  in the Write command (NVMe 2.0 spec)
 ▼
SSD FTL
 |  routes data to the correct Reclaim Group
 ▼
NAND Flash (physically separated by data temperature)
  [RG0: HOT]   [RG1: COLD]   [RG2: META]
```

Critical O_DIRECT Rules

Rule	Reason
Buffer address must be 4096-byte aligned	DMA requirement
Write size must be multiple of 4096	Device block size
File offset must be multiple of 4096	Device block size
Violating any of these → `EINVAL` error	No exception

Would you like me to extend this with a multi-threaded version, or show how to do this via `io_uring` for higher throughput?

Please show how to do this via `io_uring` for higher throughput.

FDP Write via `io_uring` for High Throughput

Why `io_uring` over `pwritev2`?

Feature	`pwritev2`	`io_uring`
Submission model	Blocking syscall per write	Batch submit, async
Syscall overhead	1 per write	1 per **batch**
CPU efficiency	Moderate	High (kernel polling mode)
Queue depth	1	Up to thousands
FDP support	✅	✅

Install Dependencies

bash

```bash
# Ubuntu/Debian
sudo apt install liburing-dev

# RHEL/Fedora
sudo dnf install liburing-devel
```

Complete Code

c

```c
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <errno.h>
#include <liburing.h>

/* ---------------------------------------------------------------
 * Configuration
 * --------------------------------------------------------------- */
#define BLOCK_SIZE      4096        /* must match device logical block size  */
#define ALIGN_SIZE      4096        /* O_DIRECT alignment requirement         */
#define QUEUE_DEPTH     64          /* io_uring submission queue depth        */
#define TOTAL_WRITES    256         /* total number of write operations       */
#define BATCH_SIZE      64          /* how many SQEs to submit per batch      */

/* FDP Placement Handle IDs */
#define PH_HOT          0
```

```c
#define PH_COLD         1
#define PH_META         2

/* RWF_PLACEMENT — upper 16 bits carry the placement handle */
#ifndef RWF_PLACEMENT
#define RWF_PLACEMENT   0x00000200
#endif

#define ENCODE_PH(ph)   (RWF_PLACEMENT | ((ph) << 16))

/* ---------------------------------------------------------------
 * Per-request context
 * Attached to each SQE so we can identify it in the CQE.
 * --------------------------------------------------------------- */
typedef struct {
    int     index;          /* request index for logging         */
    int     placement;      /* which placement handle was used   */
    off_t   offset;         /* byte offset on device             */
    void   *buf;            /* aligned data buffer               */
} write_ctx_t;

/* ---------------------------------------------------------------
 * alloc_aligned_buf
 * --------------------------------------------------------------- */
static void *alloc_aligned_buf(size_t size)
{
    void *buf = NULL;
    if (posix_memalign(&buf, ALIGN_SIZE, size) != 0) {
        perror("posix_memalign");
        return NULL;
    }
    return buf;
}

/* ---------------------------------------------------------------
 * pick_placement
 * Simple policy: round-robin hot/cold/meta based on index.
 * In a real system you'd pick based on data temperature.
 * --------------------------------------------------------------- */
static int pick_placement(int index)
{
    switch (index % 3) {
        case 0: return PH_HOT;
        case 1: return PH_COLD;
        default: return PH_META;
    }
}

/* ---------------------------------------------------------------
```

```
 * submit_batch
 * Prepares up to `batch_size` SQEs and submits them in one shot.
 *
 * Returns: number of SQEs submitted, or -1 on error.
 * ---------------------------------------------------------------- */
static int submit_batch(struct io_uring *ring,
                        int fd,
                        write_ctx_t *ctxs,
                        int start,
                        int batch_size,
                        int total)
{
    int submitted = 0;

    for (int i = start; i < start + batch_size && i < total; i++) {
        struct io_uring_sqe *sqe = io_uring_get_sqe(ring);
        if (!sqe) {
            fprintf(stderr, "SQ ring full at index %d\n", i);
            break;
        }

        write_ctx_t *ctx = &ctxs[i];

        /* Prepare the write operation */
        io_uring_prep_write(sqe, fd, ctx->buf, BLOCK_SIZE, ctx->offset);

        /* Attach FDP placement hint via ioprio field.
         * RWF_PLACEMENT tells the kernel to use FDP,
         * upper 16 bits carry the placement handle ID. */
        sqe->rw_flags = ENCODE_PH(ctx->placement);

        /* Attach our context as user_data so we can match in CQE */
        io_uring_sqe_set_data(sqe, ctx);

        submitted++;
    }

    /* Submit all prepared SQEs in ONE syscall — this is the key
     * throughput advantage over pwritev2 (one syscall per write). */
    int ret = io_uring_submit(ring);
    if (ret < 0) {
        fprintf(stderr, "io_uring_submit failed: %s\n", strerror(-ret));
        return -1;
    }

    return submitted;
}

/* ---------------------------------------------------------------
```

```c
 * harvest_completions
 * Waits for and processes `count` CQEs.
 *
 * Returns: number of failed writes.
 * ---------------------------------------------------------------- */
static int harvest_completions(struct io_uring *ring, int count)
{
    int failures = 0;

    for (int i = 0; i < count; i++) {
        struct io_uring_cqe *cqe;

        int ret = io_uring_wait_cqe(ring, &cqe);
        if (ret < 0) {
            fprintf(stderr, "io_uring_wait_cqe: %s\n", strerror(-ret));
            failures++;
            continue;
        }

        /* Retrieve our context from user_data */
        write_ctx_t *ctx = (write_ctx_t *)io_uring_cqe_get_data(cqe);

        if (cqe->res < 0) {
            fprintf(stderr, "[FAIL] idx=%-4d offset=%-10ld ph=%d → %s\n",
                    ctx->index, ctx->offset,
                    ctx->placement, strerror(-cqe->res));
            failures++;
        } else {
            printf("[OK]   idx=%-4d offset=%-10ld ph=%d (%s) → %d bytes\n",
                   ctx->index, ctx->offset,
                   ctx->placement,
                   ctx->placement == PH_HOT  ? "HOT " :
                   ctx->placement == PH_COLD ? "COLD" : "META",
                   cqe->res);
        }

        /* Mark CQE as seen — advances the CQ ring tail */
        io_uring_cqe_seen(ring, cqe);
    }

    return failures;
}

/* ---------------------------------------------------------------
 * main
 * ---------------------------------------------------------------- */
int main(void)
{
    const char *dev = "/dev/nvme0n1";
```

```c
int ret;

/* 1. Open NVMe device with O_DIRECT to bypass page cache */
int fd = open(dev, O_WRONLY | O_DIRECT);
if (fd < 0) {
    perror("open");
    return 1;
}
printf("Opened %s with O_DIRECT\n\n", dev);

/* 2. Init io_uring with SQPOLL for kernel-side polling.
 *    SQPOLL lets the kernel poll the SQ ring without a syscall,
 *    reducing overhead even further for high-throughput workloads. */
struct io_uring ring;
struct io_uring_params params = {0};
params.flags = IORING_SETUP_SQPOLL;     /* kernel polling thread     */
params.sq_thread_idle = 2000;           /* poll idle timeout: 2000ms */

ret = io_uring_queue_init_params(QUEUE_DEPTH, &ring, &params);
if (ret < 0) {
    /* SQPOLL requires CAP_SYS_NICE; fall back to normal mode */
    fprintf(stderr, "SQPOLL init failed (%s), falling back\n",
            strerror(-ret));
    ret = io_uring_queue_init(QUEUE_DEPTH, &ring, 0);
    if (ret < 0) {
        fprintf(stderr, "io_uring_queue_init: %s\n", strerror(-ret));
        close(fd);
        return 1;
    }
}

/* 3. Allocate and prepare all write contexts upfront */
write_ctx_t *ctxs = calloc(TOTAL_WRITES, sizeof(write_ctx_t));
if (!ctxs) {
    perror("calloc");
    goto cleanup;
}

for (int i = 0; i < TOTAL_WRITES; i++) {
    ctxs[i].index     = i;
    ctxs[i].placement = pick_placement(i);
    ctxs[i].offset    = (off_t)i * BLOCK_SIZE;
    ctxs[i].buf       = alloc_aligned_buf(BLOCK_SIZE);

    if (!ctxs[i].buf) goto cleanup;

    /* Fill each buffer with a pattern identifying its placement */
    memset(ctxs[i].buf, 0xA0 + ctxs[i].placement, BLOCK_SIZE);
}
```

```
    /* 4. Submit and harvest in batches */
    printf("Submitting %d writes in batches of %d...\n\n",
           TOTAL_WRITES, BATCH_SIZE);

    int total_failures = 0;

    for (int start = 0; start < TOTAL_WRITES; start += BATCH_SIZE) {
        int this_batch = (start + BATCH_SIZE < TOTAL_WRITES)
                         ? BATCH_SIZE
                         : TOTAL_WRITES - start;

        /* Submit batch */
        int submitted = submit_batch(&ring, fd, ctxs,
                                     start, this_batch, TOTAL_WRITES);
        if (submitted < 0) break;

        printf("--- Batch %d: submitted %d SQEs ---\n",
               start / BATCH_SIZE + 1, submitted);

        /* Harvest completions for this batch */
        total_failures += harvest_completions(&ring, submitted);
    }

    /* 5. Summary */
    printf("\n=== Summary ===\n");
    printf("Total writes : %d\n", TOTAL_WRITES);
    printf("Failures     : %d\n", total_failures);
    printf("Success      : %d\n", TOTAL_WRITES - total_failures);

cleanup:
    /* Free all buffers */
    if (ctxs) {
        for (int i = 0; i < TOTAL_WRITES; i++)
            free(ctxs[i].buf);
        free(ctxs);
    }

    io_uring_queue_exit(&ring);
    close(fd);
    return total_failures > 0 ? 1 : 0;
}
```

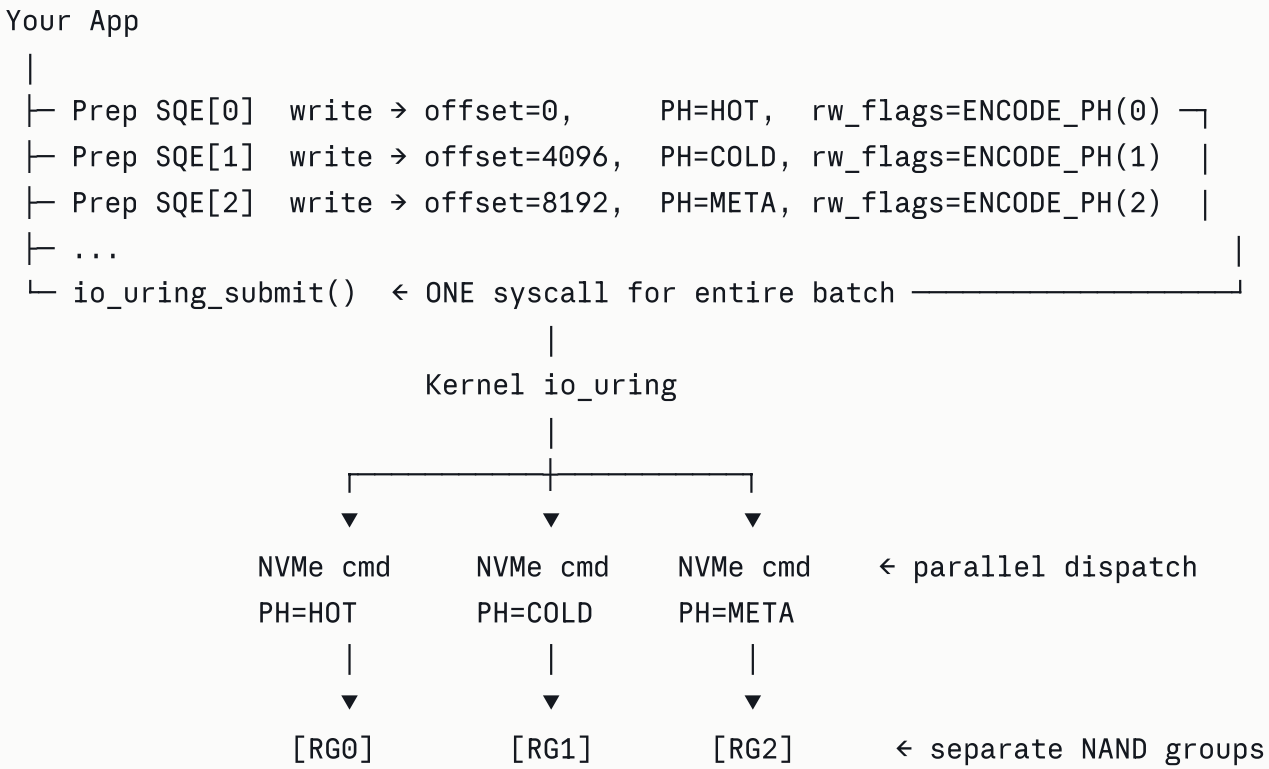
Build & Run

bash

```
# Compile — link liburing
gcc -O2 -o fdp_uring fdp_uring.c -luring

# Run as root (O_DIRECT on raw NVMe device requires it)
sudo ./fdp_uring
```

How the Batching Works



Key `io_uring` Tuning Options

Flag	Effect	When to use
`IORING_SETUP_SQPOLL`	Kernel polls SQ, zero syscall on submit	Ultra-high IOPS (>100K/s)
`IORING_SETUP_IOPOLL`	Kernel polls CQ instead of IRQ	Low-latency NVMe
`IORING_SETUP_SQPOLL` + `IORING_SETUP_IOPOLL`	Full polling, no interrupts	Maximum throughput
`io_uring_register_buffers`	Pre-register buffers with kernel	Avoids per-I/O page pinning
`io_uring_register_files`	Pre-register file descriptors	Reduces fd lookup overhead

Add Buffer & File Registration for Even More Speed

```c
/* Pre-register buffers — kernel pins them once, not per I/O */
struct iovec iovecs[TOTAL_WRITES];
for (int i = 0; i < TOTAL_WRITES; i++) {
    iovecs[i].iov_base = ctxs[i].buf;
    iovecs[i].iov_len  = BLOCK_SIZE;
}
io_uring_register_buffers(&ring, iovecs, TOTAL_WRITES);

/* Pre-register file descriptor */
io_uring_register_files(&ring, &fd, 1);

/* Then use fixed variants in SQE prep */
io_uring_prep_write_fixed(sqe, 0,           /* registered fd index */
                          ctx->buf,
                          BLOCK_SIZE,
                          ctx->offset,
                          i);               /* registered buffer index */
```

Would you like me to add performance benchmarking to compare FDP vs non-FDP write throughput?